Procedure Drift Finder: Breakdown, Approach and What Makes Us Different

Team briefing, 10 Oct 2026. Research is from public web sources; scores and opinions are my judgment. Verify before using any claim in the pitch.

1. The idea in plain English

Every factory, plant or lab has written procedures called **SOPs** (standard operating procedures). Over time, people adapt: they skip a step, swap a tool, change the order. Sometimes that is a smart fix, sometimes it is dangerous. Safety experts call the gap between the paper and reality **drift**, and when it becomes normal and nobody notices, it is called **normalization of deviation**. Major accident inquiries (Piper Alpha, an LPG explosion) point to exactly this.

The clues are usually already written down in handover notes, near-miss reports and maintenance logs. Nobody has time to read hundreds of them against the SOP. **Our tool does that reading, step by step, and shows the proof.**

2. Words you will hear

- **SOP:** the written step-by-step procedure for a task.
- **Drift:** practice slowly moving away from the written procedure.
- **Work-as-imagined vs work-as-done:** how managers think the work is done vs how it is actually done.
- **Near-miss:** something that almost caused harm but did not.
- **Handover note:** what one shift tells the next.
- **Evidence quote:** the exact words from a note that prove a drift.
- **Precision / recall:** of the flags we raise, how many are right (precision); of the real drifts, how many we find (recall).
- **Endpoint:** a web address our app sends text to and gets an AI answer back from.

3. How the system works

1. **Load the SOP** and split it into numbered steps.

2. **Load the notes** (handover notes, near-miss reports, maintenance logs).
3. **Match** each note to the SOP steps it might relate to.
4. **Judge.** The AI decides whether the note **affirmatively describes** practice that differs from the step, and answers in a fixed format: step number, drift type, exact quote, confidence.
5. **Verify.** Plain code (not AI) checks the quote really appears word for word in the note. If not, the flag is dropped. This stops made-up evidence.
6. **Aggregate.** Count flags by SOP step, shift and week, to show which steps drift most and whether it is growing.
7. **Human review.** A person approves or rejects each flag. For approved ones the tool drafts a suggested SOP fix.
8. **Benchmark.** Compare against the drifts we planted to report precision, recall and where we are wrong.

Drift types we classify:

- Skipped step
- Reordered steps
- Workaround or substitution
- **SOP is outdated or impractical** (the real fix is to change the procedure)
- **SOP is unclear** (people read it differently)

4. How we use the AMD and hackathon features

AMD technology must power a meaningful part of the working product, or we are not eligible for prizes. Here is how to do that without being experts.

Feature	What it is	How we use it	What to show judges
AMD Developer Cloud	Rent an AMD GPU in the cloud. New AMD AI Developer Program members get $100 in credits.	Start a GPU instance with a ready-made vLLM image (a past project did this) and run our model there.	A live screen of our app calling our own GPU.

Feature	What it is	How we use it	What to show judges
MI300X GPU, 192 GB memory	A very large-memory AI chip.	Run two models at once: a small fast one to match notes to steps, and a larger one to judge. Past entries ran several large models on one MI300X.	One slide: models running together on one GPU.
ROCm	AMD's software layer, the equivalent of NVIDIA's CUDA.	Nothing special to code. vLLM runs on top of it.	Name it in the architecture slide.
vLLM endpoint	A server that answers like the standard OpenAI chat API.	Our app calls it like any chat API, so models are easy to swap.	Same code, different model, in the demo.
Parallel processing	Many notes handled at once.	Process hundreds of notes in one batch.	A number: notes processed per minute.

Practical tips from past builders and common sense:

- Test the AMD setup on **Oct 12**, not Oct 17. It is a hard eligibility requirement.
- One past builder noted the cloud had no persistent storage beyond the session, so save code and data to GitHub often.
- Switch the GPU off when idle to save credits.
- Past entries used open models such as Qwen and Gemma. Test two or three and pick the one that gives the best precision.
- Past builders used Streamlit (a simple Python tool for web screens). It suits beginners.
- Evolus and Google are optional partner technologies. Skip them unless time is spare.

What the track asks for (Intelligent Industry): a working product, not only a dashboard; realistic industrial data producing a useful alert, explanation, recommendation or action; a clear user; and a measurable improvement. Our answers: a working review tool, synthetic plus public-style data, SOP-fix recommendations, the safety manager as user, and detection numbers as the improvement. Submission needs a public GitHub repository, a demo video, slides, a cover image and an app URL.

5. What already exists, and how we differ

Project or product	What it does	How we differ
AndonAI (ACT II hackathon entry)	Incident command board. Takes images, sensor logs, shift notes and SOP documents, then agents match an issue to a procedure, suggest causes and write an audit-ready incident report.	It handles one incident at a time. We read **routine, non-incident notes** for patterns across steps, shifts and weeks, and we propose SOP fixes.
SENTRA (hackathon entry)	Predictive maintenance on machine sensor data.	Different data: sensors vs words. Complementary, not competing.
AsBuilt Lens (hackathon entry)	Visual verification of what exists vs what should.	Image-based. Ours is text-based.
Intenseye, Protex AI and other camera tools	Detect unsafe acts and missed steps from video.	Need cameras and monitor people. We use text the plant already writes.
Taqtics and other audit software	Auditors fill SOP checklists and the tool scores compliance by line, shift and revision.	Depends on auditors. We mine existing notes automatically.
iFactory and QMS deviation software	Log reported deviations, link corrective actions, show trends.	Starts from formal reports. We find the **unreported** drift in informal notes.
Poka, XMPro and handover tools	Create structured handover reports and surface SOPs to operators.	They improve the handover. We check practice against the SOP.
Generative AI for SOP drafting	Drafts SOP updates from approved process-change information.	We draft SOP updates from **observed practice**, for human approval.

6. What makes us stand out (say these to the team)

1. **We read routine notes, not only incidents.** Most tools react when something goes wrong. We look for drift before it does.
2. **Step-level proof.** Every flag points to the exact SOP step and the exact quote, and plain code confirms the quote is real.
3. **Drift is not always a violation.** We tell the user when the **procedure itself** is the problem, so the answer can be "fix the SOP", not just "retrain the worker".
4. **Trends, not just alerts.** A heat strip of which steps drift most, on which shifts, growing or not.
5. **About procedures, not people.** We report on steps and shifts, never score individuals. No cameras, no new monitoring.
6. **Honest measured results.** We plant known drifts and report precision, recall, quote-verification rate, our false alarms and throughput on AMD hardware.

7. What we must NOT claim

- **Not "first".** Camera tools, audit tools and a hackathon entry (AndonAI) overlap with parts of this.
- **Silence is not evidence.** Research shows LLM judges are poor at noticing what is missing. We only flag drift when a note **says** something ("we usually skip the second check", "used a wrench instead"). If a step is simply not mentioned, we say nothing.
- **Not a replacement for safety controls.** Every flag needs a human decision.
- **Synthetic data.** Say so upfront. Mix in messy, realistic notes and public incident narratives (for example OSHA's published severe injury reports; verify access).

8. The 3-minute demo story

1. **The gap (20 sec):** show one SOP step and one real-sounding note that contradicts it.
2. **The load (20 sec):** feed in 200 notes on the AMD GPU and show notes per minute.
3. **The result (60 sec):** the heat strip, then click the top drift to see the SOP step, the quote and the shifts affected.
4. **The twist (30 sec):** open a flag classified "SOP is impractical" and show the suggested fix.
5. **The proof (30 sec):** benchmark numbers against 25 planted drifts.

6. **The honesty slide (20 sec):** "Where our AI is wrong" with real false alarms.

9. How to build the test data (most important job)

- Write 3 to 5 SOPs (for example lockout-tagout, forklift battery charging, confined-space entry). An AI assistant can draft them.
- Plant **25 known drifts**, each as a short fact ("night shift skips the second isolation check").
- Generate about 200 notes in varied styles, lengths and tones: some mention drifts, some are clean, and some are **traps** (mention the same words but follow the procedure).
- Keep an answer key listing which note contains which drift. This is how we score ourselves.
- Stretch: add notes in other languages if the team speaks them.

10. Roles and week plan

Roles (6 people): Lead; Data lead (SOPs, notes, answer key); Builder A (AI pipeline and verification); Builder B (aggregation, trends, benchmark script); Front-end (Streamlit screens); Pitch and demo (README, slides, video).

Day	Goal
Oct 12	Sign up, test the AMD GPU and vLLM, create the repository, design the data
Oct 13	Generate SOPs and notes with answer key. First working pipeline on 10 notes
Oct 14	Quote verification, drift types, run on all 200 notes
Oct 15	Aggregation and trends, review screen, SOP-fix drafts
Oct 16	Benchmark, fix worst errors, **freeze features**
Oct 17	Polish, record the demo, submit
Oct 18	Buffer, last fixes, on-site pitch if approved. Confirm the exact submission deadline

If we fall behind, cut in this order: other languages, SOP-fix drafts, the second model, trend charts. **Never cut:** quote verification, the answer key and the benchmark.

11. Open questions for the team

1. Which languages does the team speak, and does anyone know a plant or lab for realistic notes?
2. Who is lead, data lead, and who builds the screen?
3. Which industry setting do we use for the demo (manufacturing, warehouse, lab)?